

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class Inventory : MonoBehaviour
5 {
6     public int maxSlot = 10;
7     private List<string> items = new List<string>();
8
9     public bool AddItem(string itemName)
10    {
11        if (items.Count < maxSlot)
12        {
13            items.Add(itemName);
14            return true;
15        }
16
17        return false;
18    }
19
20    public void RemoveItem(string itemName)
21    {
22        items.Remove(itemName);
23    }
24
25    public List<string> GetItems()
26    {
27        return new List<string>(items);
28    }
29 }=

```

```

1 using UnityEngine;
2
3 public enum ElementType
4 {
5     Fire,
6     Water,
7     Earth
8 }
9
10 public class DamageCalculator : MonoBehaviour
11 {
12     private float GetElementMultiplier(
13         ElementType attacker,
14         ElementType defender)
15     {
16         if (attacker == defender)
17             return 1.0f;
18
19         bool isSuperEffective =
20             (attacker == ElementType.Fire && defender == ElementType.Earth) ||
21             (attacker == ElementType.Earth && defender == ElementType.Water) ||
22             (attacker == ElementType.Water && defender == ElementType.Fire);
23
24         return isSuperEffective ? 1.5f : 0.5f;
25     }
26
27     public int CalculateDamage(
28         int baseDamage,
29         ElementType attackerType,
30         ElementType defenderType,
31         int defenderDefense)
32     {
33         float damage = baseDamage;
34
35         damage *= GetElementMultiplier(
36             attackerType,
37             defenderType);
38
39         bool isCritical = Random.Range(0f, 1f) < 0.2f;
40
41         if (isCritical)
42         {
43             damage *= 2f;
44         }
45
46         damage -= defenderDefense;
47
48         int finalDamage = Mathf.Max(
49             1,
50             Mathf.RoundToInt(damage));
51
52         return finalDamage;
53     }
54 }
55
56 Tab to adapt code Esc Dismiss

```

```

1 using UnityEngine;
2
3 public abstract class GameCharacter : MonoBehaviour
4 {
5     public string namaKarakter;
6     public int hpMaksimum;
7     protected int hpSaatIni;
8
9     protected virtual void Awake()
10    {
11        hpSaatIni = hpMaksimum;
12    }
13
14    public void TakeDamage(int dmg)
15    {
16        hpSaatIni -= dmg;
17
18        if (hpSaatIni < 0)
19            hpSaatIni = 0;
20    }
21
22    public bool IsAlive()
23    {
24        return hpSaatIni > 0;
25    }
26
27    public abstract void Attack();
28 }
29

```

```

1 using UnityEngine;
2
3 public class Player : GameCharacter
4 {
5     public int exp = 0;
6     public int level = 1;
7
8     public override void Attack()
9     {
10        Debug.Log(namaKarakter + " menyerang menggunakan senjata!");
11    }
12
13    public void GainExp(int jumlahExp)
14    {
15        exp += jumlahExp;
16
17        if (exp >= level * 100)
18        {
19            LevelUp();
20        }
21    }
22
23    private void LevelUp()
24    {
25        level++;
26        hpMaksimum += 20;
27
28        Debug.Log(
29            namaKarakter +
30            " naik ke Level " +
31            level +
32            "!");
33    }
34 }
35

```

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class EnemySpawner : MonoBehaviour
5 {
6     private GameObject player;
7     private Rigidbody playerRb;
8
9     private List<EnemyAI> enemyAICollection =
10    new List<EnemyAI>();
11
12    void Start()
13    {
14        player = GameObject.Find("Player");
15
16        playerRb = player.GetComponent<Rigidbody>();
17
18        GameObject[] enemies =
19            GameObject.FindGameObjectsWithTag("Enemy");
20
21        foreach (GameObject enemy in enemies)
22        {
23            enemyAICollection.Add(
24                enemy.GetComponent<EnemyAI>());
25        }
26
27    }
28
29    void Update()
30    {
31        foreach (EnemyAI enemyAI in enemyAICollection)
32        {
33            if (enemyAI == null)
34                continue;
35
36            float distance = Vector3.Distance(
37                enemyAI.transform.position,
38                player.transform.position);
39
40            if (distance < 5f)
41            {
42                enemyAI.ChasePlayer(playerRb);
43            }
44        }
45    }
46 }
47

```

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class SpatialHashGrid
5 {
6     private readonly float cellSize;
7
8     private readonly Dictionary<int, int, List<GameObject>> cells =
9         new Dictionary<int, int, List<GameObject>>();
10
11     public SpatialHashGrid(float cellSize)
12     {
13         this.cellSize = cellSize;
14     }
15
16     private (int, int) GetCellCoord(Vector3 pos)
17     {
18         int cx = Mathf.FloorToInt(pos.x / cellSize);
19         int cy = Mathf.FloorToInt(pos.z / cellSize);
20
21         return (cx, cy);
22     }
23
24     public void Clear()
25     {
26         cells.Clear();
27     }
28
29     public void Insert(GameObject obj)
30     {
31         var coord = GetCellCoord(
32             obj.transform.position);
33
34         if (!cells.TryGetValue(
35             coord,
36             out var list))
37         {
38             list = new List<GameObject>();
39             cells[coord] = list;
40         }
41
42         list.Add(obj);
43     }
44
45     public List<GameObject> GetNearbyObjects(
46         GameObject obj)
47     {
48         var result =
49             new List<GameObject>();
50
51         var (cx, cy) =
52             GetCellCoord(
53                 obj.transform.position);
54
55         for (int dx = -1; dx <= 1; dx++)
56         {
57             for (int dy = -1; dy <= 1; dy++)
58             {
59                 if (cells.TryGetValue(
60                     (cx + dx, cy + dy),
61                     out var list))
62                 {
63                     result.AddRange(list);
64                 }
65             }
66         }
67
68         return result;
69     }
70 }
71
72

```

```

1 using System.Threading.Tasks;
2 using UnityEngine;
3
4 public class PlayerData
5 {
6     public int gold;
7     public int level;
8 }
9
10 public class SaveSystem : MonoBehaviour
11 {
12     private PlayerData data =
13         new PlayerData
14         {
15             gold = 0,
16             level = 1
17         };
18
19     public void AddGold(int amount)
20     {
21         data.gold += amount;
22     }
23
24     public void SaveToFileAsync()
25     {
26         PlayerData snapshot =
27             new PlayerData
28             {
29                 gold = data.gold,
30                 level = data.level
31             };
32
33         Task.Run(() =>
34         {
35             string json =
36                 JsonUtility.ToJson(snapshot);
37
38             System.IO.File.WriteAllText(
39                 "save.json",
40                 json);
41         });
42     }
43 }
44

```

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class ObjectPool<T> where T : Component
5 {
6     private readonly Queue<T> pool =
7         new Queue<T>();
8
9     private readonly T prefab;
10    private readonly Transform parent;
11
12    public ObjectPool(
13        T prefab,
14        int prewarmCount,
15        Transform parent = null)
16    {
17        this.prefab = prefab;
18        this.parent = parent;
19
20        for (int i = 0; i < prewarmCount; i++)
21        {
22            T obj =
23                Object.Instantiate(
24                    prefab,
25                    parent);
26
27            obj.gameObject.SetActive(false);
28            pool.Enqueue(obj);
29        }
30
31    }
32
33    public T Get()
34    {
35        T obj =
36            pool.Count > 0
37            ? pool.Dequeue()
38            : Object.Instantiate(
39                prefab,
40                parent);
41
42        obj.gameObject.SetActive(true);
43
44        (obj as IPoolable)?
45            .OnSpawnFromPool();
46
47        return obj;
48    }
49
50    public void Release(T obj)
51    {
52        (obj as IPoolable)?
53            .OnReturnToPool();
54
55        obj.gameObject.SetActive(false);
56        pool.Enqueue(obj);
57    }
58 }
59
60 public interface IPoolable
61 {
62     void OnSpawnFromPool();
63     void OnReturnToPool();
64 }
65
66
67

```